

HW02

ENAE 601: Astrodynamics

September 30th. at 23:59

Dr. Healy

Vai Srivastava

Problem 1

Where Curtis says “orbital elements”, he substitutes angular momentum for the standard semi-major axis. Show how to compute one from the other.

For the following problems and the rest of the semester, compute semi-major axis instead of angular momentum when classical (Keplerian) orbit elements are requested, unless otherwise specified.

We want functions $f(h) = a$ and $g(a) = h$ for specific angular momentum $h = |\mathbf{h}|$ and semi-major axis a .

Semi-latus rectum p relates to semi-major axis by:

$$p = a(1 - e^2)$$

and to specific angular momentum by:

$$p = \frac{h^2}{\mu}$$

Equating the two gives:

$$a(1 - e^2) = \frac{h^2}{\mu}$$

Solving for h gives:

$$h = \sqrt{\mu a(1 - e^2)}$$

and solving for a gives:

$$a = \frac{h^2}{\mu(1 - e^2)}$$

Thus, we have (as long as $e \neq 1$):

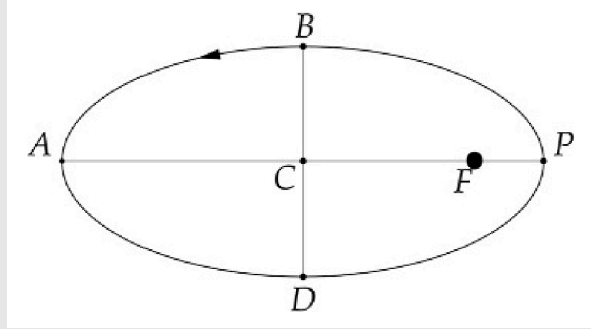
$$h = \sqrt{\mu a(1 - e^2)}$$

$$a = \frac{h^2}{\mu(1 - e^2)} \quad \blacksquare$$

Problem 2

Curtis 3.5

Calculate the time required to fly from P to B , in terms of the eccentricity e and the period T . B lies on the minor axis.



The time since periapsis for an elliptical orbit, given the mean anomaly and the period is:

$$M_e = \frac{2\pi}{T}t$$

The relation between the mean anomaly and the eccentric anomaly for an elliptical orbit, given the eccentricity, is:

$$M_e = E - e \sin(E)$$

Equating the two gives:

$$\frac{2\pi}{T}t = E - e \sin(E)$$

Solving for t gives:

$$t = \frac{E - e \sin(E)}{2\pi}T$$

As B is on the minor axis, $E = \frac{\pi}{2}$.

Substituting $E = \frac{\pi}{2}$ and computing:

$$\begin{aligned} t &= \frac{E - e \sin(E)}{2\pi}T \\ &= \frac{\frac{\pi}{2} - e \sin(\frac{\pi}{2})}{2\pi}T \\ &= \left(\frac{1}{4} - \frac{e}{2\pi}\right)T \end{aligned}$$

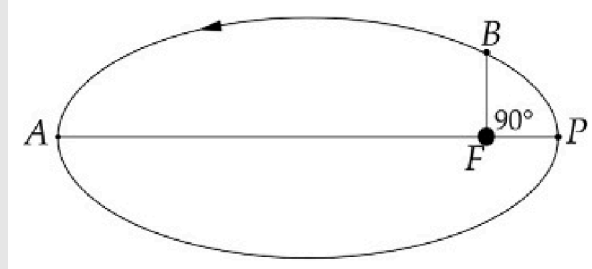
Thus, the time required is:

$$t = \left(\frac{1}{4} - \frac{e}{2\pi}\right)T \quad \blacksquare$$

Problem 3

Curtis 3.6

If the eccentricity of the elliptical orbit is 0.3, calculate, in terms of the period T , the time required to fly from P to B .



Using our formula for the time since periapsis for an elliptical orbit, given the eccentric anomaly, eccentricity, and the period from our solution to Problem 2:

$$t = \frac{E - e \sin(E)}{2\pi} T$$

The relation between eccentric anomaly and true anomaly is:

$$E = \arccos\left(\frac{e + \cos(\theta)}{1 + e \cos(\theta)}\right)$$

Combining these equations, we have:

$$t = \frac{\arccos\left(\frac{e + \cos(\theta)}{1 + e \cos(\theta)}\right) - e \sin\left(\arccos\left(\frac{e + \cos(\theta)}{1 + e \cos(\theta)}\right)\right)}{2\pi} T$$

As the angle between B and P is 90° , so $\theta = \frac{\pi}{2}$.

Substituting $e = 0.3$ and $\theta = \frac{\pi}{2}$ and computing:

$$\begin{aligned} t &= \frac{\arccos\left(\frac{e + \cos(\theta)}{1 + e \cos(\theta)}\right) - e \sin\left(\arccos\left(\frac{e + \cos(\theta)}{1 + e \cos(\theta)}\right)\right)}{2\pi} T \\ &= \frac{\arccos\left(\frac{0.3 + \cos(\frac{\pi}{2})}{1 + 0.3 \cos(\frac{\pi}{2})}\right) - 0.3 \sin\left(\arccos\left(\frac{0.3 + \cos(\frac{\pi}{2})}{1 + 0.3 \cos(\frac{\pi}{2})}\right)\right)}{2\pi} T \\ &= 0.155959T \end{aligned}$$

Thus, the time required is:

$$t = 0.155959T \approx 0.1560T \quad \blacksquare$$

Problem 4**Curtis 3.8**

A satellite is in Earth orbit for which the perigee altitude is 200 km and the apogee altitude is 600 km. Find the time interval during which the satellite remains above an altitude of 400 km.

Take $\mu = 398\,600 \text{ km}^3 \text{ s}^{-2}$ and $R_E = 6378 \text{ km}$. The apsidal radii, semi-major axis, and eccentricity are

$$r_p = R_E + 200 \text{ km} = 6578 \text{ km},$$

$$r_a = R_E + 600 \text{ km} = 6978 \text{ km},$$

$$a = \frac{r_p + r_a}{2} = 6778 \text{ km},$$

$$e = \frac{r_a - r_p}{r_a + r_p} = 0.0295072.$$

Thus, the limiting altitude of 400 km corresponds to $r = 6778 \text{ km} = a$. From $r = a(1 - e \cos(E))$, the outbound and inbound crossings occur at $E_1 = \frac{\pi}{2}$ and $E_2 = 3\frac{\pi}{2}$. Using $M_e = E - e \sin(E)$ and $n = \sqrt{\frac{\mu}{a^3}}$,

$$\begin{aligned} \Delta t &= \frac{(E_2 - e \sin(E_2)) - (E_1 - e \sin(E_1))}{n} \\ &= \frac{\pi + 2e}{n} \\ &= 2828.89 \text{ s} \\ &= 47.1482 \text{ min.} \end{aligned}$$

Therefore, the satellite remains above 400 km for

$$\Delta t = 47.15 \text{ min} \quad \blacksquare$$

Problem 5**Curtis 3.9**

An Earth-orbiting satellite has a perigee radius of 7000 km and an apogee radius of 10000 km.

- What true anomaly $\Delta\theta$ is swept out between $t = 0.5$ h and $t = 1.5$ h after perigee passage?
- What area is swept out by the position vector during that time interval?

- The orbit parameters and mean motion are

$$a = \frac{r_p + r_a}{2} = 8500 \text{ km},$$

$$e = \frac{r_a - r_p}{r_a + r_p} = 0.1764706,$$

$$n = \sqrt{\frac{\mu}{a^3}}.$$

At each time, solve Kepler's equation $M_e = nt = E - e \sin(E)$, then obtain the true anomaly from

$$\theta = 2 \arctan \left(\sqrt{\frac{1+e}{1-e}} \tan \left(\frac{E}{2} \right) \right).$$

Taking the quadrant of θ from E gives

$$\begin{aligned} \theta(0.5 \text{ h}) &= 103.2819^\circ, \\ \theta(1.5 \text{ h}) &= 231.9863^\circ. \end{aligned}$$

Hence,

$$\Delta\theta = 128.7044^\circ \quad \blacksquare$$

- Kepler's second law gives a constant areal velocity $\dot{A} = \frac{h}{2}$. Here

$$h = \sqrt{\mu a(1 - e^2)} = 57293.8761 \text{ km}^2 \text{ s}^{-1}.$$

Since $\Delta t = 1 \text{ h} = 3600 \text{ s}$,

$$\begin{aligned} \Delta A &= \frac{h}{2} \Delta t \\ &= 103\,128\,977 \text{ km}^2. \end{aligned}$$

Therefore, the swept area is

$$\Delta A = 103\,130\,000 \text{ km}^2 \quad \blacksquare$$

Problem 6**Curtis 3.10**

An Earth-orbiting satellite has a period of 14 h and a perigee radius of 10000 km. At time $t = 10$ h after perigee passage, determine:

- The radial position.
- The speed.
- The radial component of the velocity.

- a) From Kepler's third law and the perigee condition,

$$a = \sqrt[3]{\mu \left(\frac{T}{2\pi} \right)^2} = 29490.324 \text{ km},$$

$$e = 1 - \frac{r_p}{a} = 0.6609057.$$

Ten hours after perigee, $M_e = 2\pi \frac{t}{T} = 10\frac{\pi}{7}$. Solving $M_e = E - e \sin(E)$ gives $E = 228.6963^\circ$. Therefore,

$$r = a(1 - e \cos(E))$$

$$= 42354.921 \text{ km}.$$

Thus,

$$r = 42355 \text{ km} \quad \blacksquare$$

- b) The vis-viva equation gives

$$v = \sqrt{\mu \left(\frac{2}{r} - \frac{1}{a} \right)}$$

$$= 2.303389 \text{ km s}^{-1}.$$

Hence,

$$v = 2.3034 \text{ km s}^{-1} \quad \blacksquare$$

- c) Differentiating $r = a(1 - e \cos(E))$ and using $\dot{E} = \frac{n}{1 - e \cos(E)}$ gives

$$v_r = \dot{r} = \frac{nae \sin(E)}{1 - e \cos(E)}.$$

Substitution yields

$$v_r = -1.27090 \text{ km s}^{-1} \quad \blacksquare$$

The negative sign is expected because the spacecraft is moving inward after apogee.

Problem 7**Curtis 3.15**

A spacecraft on a parabolic trajectory around the Earth has a perigee radius of 6600 km.

- a) How long does it take to coast from $\theta = -90^\circ$ to $\theta = +90^\circ$?
- b) How far is the spacecraft from the center of the Earth 36 h after passing through perigee?

- a) For a parabola, $p = 2r_p = 13200$ km. Barker's equation is

$$t - t_p = \frac{1}{2} \sqrt{\frac{p^3}{\mu}} \left(D + \frac{D^3}{3} \right), \quad D = \tan\left(\frac{\theta}{2}\right).$$

At $\theta = \pm\frac{\pi}{2}$, $D = \pm 1$. Symmetry about perigee therefore gives

$$\begin{aligned} \Delta t &= \sqrt{\frac{p^3}{\mu}} \left(1 + \frac{1}{3} \right) \\ &= 3202.809 \text{ s} \\ &= 53.3801 \text{ min.} \end{aligned}$$

Thus,

$$\Delta t = 53.38 \text{ min} \quad \blacksquare$$

- b) At $t - t_p = 36$ h, Barker's equation becomes

$$D + \frac{D^3}{3} = \frac{2t}{\sqrt{\frac{p^3}{\mu}}}.$$

Its real solution is $D = 6.7206602$. Since $r = \frac{p}{1 + \cos(\theta)} = r_p(1 + D^2)$,

$$\begin{aligned} r &= 6600 \text{ km}(1 + 6.7206602^2) \\ &= 304\,704.005 \text{ km.} \end{aligned}$$

Therefore,

$$r = 304\,704 \text{ km} \quad \blacksquare$$

Problem 8**Curtis 3.16**

A spacecraft on a hyperbolic trajectory around the Earth has a perigee radius of 6600 km and a perigee speed of $1.2v_{\text{esc}}$.

- a) How long does it take to coast from $\theta = -90^\circ$ to $\theta = +90^\circ$?
- b) How far is the spacecraft from the center of the Earth 24 h after passing through perigee?

- a) At perigee, $v_p^2 = \frac{\mu(1+e)}{r_p}$, while $v_{\text{esc}}^2 = 2\frac{\mu}{r_p}$. Consequently,

$$e = 2(1.2)^2 - 1 = 1.88, \quad |a| = \frac{r_p}{e-1} = 7500 \text{ km.}$$

For a hyperbola,

$$\begin{aligned} M_h &= e \sinh(F) - F = n(t - t_p), \\ \tanh\left(\frac{F}{2}\right) &= \sqrt{\frac{e-1}{e+1}} \tanh\left(\frac{\theta}{2}\right), \\ n &= \sqrt{\frac{\mu}{|a|^3}}. \end{aligned}$$

Evaluating M_h at $\theta = 90^\circ$ and using symmetry about perigee yields

$$\Delta t = 3597.03 \text{ s} = 59.9504 \text{ min} \quad \blacksquare$$

- b) At $t - t_p = 24 \text{ h}$, $M_h = nt$. Solving $e \sinh(F) - F = M_h$, then using $r = |a|(e \cosh(F) - 1)$, gives

$$r = 656\,610.72 \text{ km} \quad \blacksquare$$

Problem 9**Curtis 3.17**

A trajectory has a perigee velocity $1.1v_{\text{esc}}$ and a perigee altitude of 200 km. If at 10 a.m., the satellite is travelling towards the Earth with a speed of 8 km s^{-1} , how far will it be from the Earth's surface at 5 p.m. the same day?

The orbit's perigee radius is $r_p = R_E + 200 \text{ km} = 6578 \text{ km}$. As in Problem 8,

$$e = 2(1.1)^2 - 1 = 1.42,$$

$$|a| = \frac{r_p}{e - 1} = 15661.905 \text{ km}.$$

At 10 a.m., the vis-viva equation gives

$$r_0 = \frac{2}{\frac{v_0^2}{\mu} - \frac{1}{|a|}} = 20679.792 \text{ km}.$$

Because the spacecraft is inbound, choose the negative solution of

$$r_0 = |a|(e \cosh(F_0) - 1),$$

which gives $F_0 < 0$ and $M_{h,0} = e \sinh(F_0) - F_0$. This state is 0.656 586 h before perigee. Propagating seven hours to 5 p.m.,

$$M_{h,f} = M_{h,0} + \sqrt{\frac{\mu}{|a|^3}} 7 \text{ h},$$

and solving hyperbolic Kepler's equation gives

$$r_f = |a|(e \cosh(F_f) - 1) = 142\,628.717 \text{ km}.$$

The requested distance from Earth's surface is therefore

$$h_f = r_f - R_E = 136\,250.72 \text{ km} \quad \blacksquare$$

Problem 10**Curtis 3.18**

An incoming object is sighted at an altitude of 100 000 km with a speed of 6 km s^{-1} and a flight path angle of -80° .

- Will it impact the Earth or fly by?
- What is the time to impact or to closest approach?

- a) At the sighting, $r_0 = R_E + 100\,000 \text{ km} = 106\,378 \text{ km}$. The specific angular momentum and energy are

$$h = r_0 v_0 \cos(\gamma_0),$$

$$\mathcal{E} = \frac{v_0^2}{2} - \frac{\mu}{r_0}.$$

The orbital parameters are then

$$e = \sqrt{1 + \frac{2\mathcal{E}h^2}{\mu^2}} = 1.7899667,$$

$$p = \frac{h^2}{\mu},$$

$$r_p = \frac{p}{1+e} = 11046.133 \text{ km}.$$

Since $r_p > R_E$, the object misses Earth. Its closest-approach altitude is

$$r_p - R_E = 4668.133 \text{ km}.$$

Thus, it will

fly by Earth ■

- b) The hyperbola has

$$|a| = \frac{\mu}{2\mathcal{E}} = 13983.036 \text{ km}.$$

Obtain the incoming true anomaly from

$$\cos(\theta_0) = \frac{\frac{p}{r_0} - 1}{e},$$

choosing $\theta_0 < 0$ because the object is approaching Earth. Then

$$F_0 = 2 \tanh^{-1} \left(\sqrt{\frac{e-1}{e+1}} \tan\left(\frac{\theta_0}{2}\right) \right),$$

$$M_{h,0} = e \sinh(F_0) - F_0.$$

Perigee corresponds to $M_h = 0$, so the remaining time is

$$\Delta t = -\frac{M_{h,0}}{\sqrt{\frac{\mu}{|a|^3}}} = 16150.953 \text{ s} = 4.486 \text{ h}.$$

Therefore,

$$\Delta t = 4.486 \text{ h} \quad \blacksquare$$

Code

```

./src/index.py Python
1  """Numerical calculations for ENAE 601 Homework 02."""
2
3  from pathlib import Path
4
5  import numpy as np
6  from scipy.optimize import brentq
7
8
9  MU_EARTH = 398600.0 # km^3/s^2
10 R_EARTH = 6378.0 # km
11 OUTPUT_DIR = Path(__file__).resolve().parents[1] / "outputs" / "text"
12
13
14 def heading(problem: str) -> None:
15     """Print the heading used to separate each problem's output."""
16     print(f"\n-----\n {problem}\n-----")
17
18
19 def write_output(problem: str, lines: list[str]) -> None:
20     """Print and save a problem's text output."""
21     output = "\n".join(lines) + "\n"
22     print(output, end="")
23     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
24     (OUTPUT_DIR / f"s{problem}.txt").write_text(output, encoding="utf-8")
25
26
27 def eccentric_anomaly(mean_anomaly: float, eccentricity: float) -> float:
28     """Solve elliptic Kepler's equation for E in [0, 2 pi]."""
29     return brentq(
30         lambda anomaly: anomaly - eccentricity * np.sin(anomaly) - mean_anomaly,
31         0.0,
32         2.0 * np.pi,
33     )
34
35
36 def hyperbolic_anomaly(mean_anomaly: float, eccentricity: float) -> float:
37     """Solve hyperbolic Kepler's equation; works for either sign of M."""
38     extent = max(1.0, np.arcsinh(abs(mean_anomaly) / eccentricity) + 1.0)
39     return brentq(
40         lambda anomaly: eccentricity * np.sinh(anomaly) - anomaly - mean_anomaly,
41         -extent,
42         extent,
43     )
44

```

```

45
46 def hyperbolic_mean_from_true(true_anomaly: float, eccentricity: float) -> float:
47     """Return signed hyperbolic mean anomaly from signed true anomaly."""
48     factor = np.sqrt((eccentricity - 1.0) / (eccentricity + 1.0))
49     anomaly = 2.0 * np.arctanh(factor * np.tan(true_anomaly / 2.0))
50     return eccentricity * np.sinh(anomaly) - anomaly
51
52
53 def problem_02() -> None:
54     """Curtis 3.5: flight time from periapsis to the minor axis."""
55     heading("p02")
56     eccentric_anomaly = np.pi / 2.0
57     constant_term = eccentric_anomaly / (2.0 * np.pi)
58     eccentricity_coefficient = -np.sin(eccentric_anomaly) / (2.0 * np.pi)
59     write_output(
60         "02",
61         [
62             f"E_B = {eccentric_anomaly:.12f} rad = pi/2",
63             f"t/T = (E_B - e sin(E_B))/(2 pi)",
64             f"t/T = {constant_term:.12f} "
65             f"{eccentricity_coefficient:+.12f} e",
66             "exact result: t/T = 1/4 - e/(2 pi)",
67         ],
68     )
69
70
71 def problem_03() -> None:
72     """Curtis 3.6: flight time to a true anomaly of 90 degrees."""
73     heading("p03")
74     eccentricity = 0.3
75     true_anomaly = np.pi / 2.0
76     eccentric_anomaly_value = np.arccos(
77         (eccentricity + np.cos(true_anomaly))
78         / (1.0 + eccentricity * np.cos(true_anomaly))
79     )
80     time_period_ratio = (
81         eccentric_anomaly_value
82         - eccentricity * np.sin(eccentric_anomaly_value)
83     ) / (2.0 * np.pi)
84     write_output(
85         "03",
86         [
87             f"e = {eccentricity:.12f}",
88             f"theta_B = {np.degrees(true_anomaly):.12f} deg",
89             f"E_B = {eccentric_anomaly_value:.12f} rad "
90             f"= {np.degrees(eccentric_anomaly_value):.12f} deg",

```

```

91         f"t/T = {time_period_ratio:.12f}",
92     ],
93 )
94
95
96 def problem_04() -> None:
97     """Curtis 3.8: duration above 400 km altitude."""
98     heading("p04")
99     radius_perigee = R_EARTH + 200.0
100    radius_apogee = R_EARTH + 600.0
101    semimajor_axis = (radius_perigee + radius_apogee) / 2.0
102    eccentricity = (radius_apogee - radius_perigee) / (
103        radius_apogee + radius_perigee
104    )
105    mean_motion = np.sqrt(MU_EARTH / semimajor_axis**3)
106    eccentric_anomaly_crossing = np.arccos(
107        (1.0 - (R_EARTH + 400.0) / semimajor_axis) / eccentricity
108    )
109    mean_anomaly_crossing = eccentric_anomaly_crossing - eccentricity * np.sin(
110        eccentric_anomaly_crossing
111    )
112    duration = (2.0 * np.pi - 2.0 * mean_anomaly_crossing) / mean_motion
113    write_output(
114        "04",
115        [
116            f"a = {semimajor_axis:.6f} km",
117            f"e = {eccentricity:.9f}",
118            f"time above 400 km = {duration:.6f} s = {duration / 60.0:.6f} min",
119        ],
120    )
121
122
123 def problem_05() -> None:
124     """Curtis 3.9: swept anomaly and area over a time interval."""
125     heading("p05")
126     radius_perigee, radius_apogee = 7000.0, 10000.0
127     semimajor_axis = (radius_perigee + radius_apogee) / 2.0
128     eccentricity = (radius_apogee - radius_perigee) / (
129         radius_apogee + radius_perigee
130     )
131     mean_motion = np.sqrt(MU_EARTH / semimajor_axis**3)
132     times = np.array([0.5, 1.5]) * 3600.0
133     mean_anomalies = mean_motion * times
134     eccentric_anomalies = np.array(
135         [eccentric_anomaly(mean_anomaly, eccentricity) for mean_anomaly in mean_anomalies]
136     )

```

```

137     true_anomalies = 2.0 * np.arctan2(
138         np.sqrt(1.0 + eccentricity) * np.sin(eccentric_anomalies / 2.0),
139         np.sqrt(1.0 - eccentricity) * np.cos(eccentric_anomalies / 2.0),
140     )
141     delta_true_anomaly = true_anomalies[1] - true_anomalies[0]
142     angular_momentum = np.sqrt(
143         MU_EARTH * semimajor_axis * (1.0 - eccentricity**2)
144     )
145     swept_area = 0.5 * angular_momentum * (times[1] - times[0])
146     common = [
147         f"a = {semimajor_axis:.6f} km; e = {eccentricity:.9f}",
148         f"theta(0.5 h) = {np.degrees(true_anomalies[0]):.6f} deg",
149         f"theta(1.5 h) = {np.degrees(true_anomalies[1]):.6f} deg",
150     ]
151     write_output(
152         "05a",
153         common + [f"delta theta = {np.degrees(delta_true_anomaly):.6f} deg"],
154     )
155     write_output(
156         "05b",
157         [
158             f"swept area = {swept_area:.6f} km^2",
159         ],
160     )
161
162
163 def problem_06() -> None:
164     """Curtis 3.10: state after 10 hours on an elliptic orbit."""
165     heading("p06")
166     period = 14.0 * 3600.0
167     radius_perigee = 10000.0
168     semimajor_axis = np.cbrt(MU_EARTH * (period / (2.0 * np.pi)) ** 2)
169     eccentricity = 1.0 - radius_perigee / semimajor_axis
170     mean_motion = 2.0 * np.pi / period
171     mean_anomaly = mean_motion * 10.0 * 3600.0
172     anomaly = eccentric_anomaly(mean_anomaly, eccentricity)
173     radius = semimajor_axis * (1.0 - eccentricity * np.cos(anomaly))
174     speed = np.sqrt(MU_EARTH * (2.0 / radius - 1.0 / semimajor_axis))
175     radial_speed = (
176         mean_motion
177         * semimajor_axis
178         * eccentricity
179         * np.sin(anomaly)
180         / (1.0 - eccentricity * np.cos(anomaly))
181     )
182     common = [

```

```

183         f"a = {semimajor_axis:.6f} km; e = {eccentricity:.9f}",
184         f"E(10 h) = {np.degrees(anomaly):.6f} deg",
185     ]
186     write_output("06a", common + [f"radius = {radius:.6f} km"])
187     write_output("06b", [f"speed = {speed:.9f} km/s"])
188     write_output(
189         "06c",
190         [
191             f"radial velocity = {radial_speed:.9f} km/s",
192         ],
193     )
194
195
196 def problem_07() -> None:
197     """Curtis 3.15: parabolic time of flight and 36-hour radius."""
198     heading("p07")
199     radius_perigee = 6600.0
200     semilatus_rectum = 2.0 * radius_perigee
201     barker_scale = 0.5 * np.sqrt(semilatus_rectum**3 / MU_EARTH)
202     time_minus_90_to_plus_90 = 2.0 * barker_scale * (1.0 + 1.0 / 3.0)
203     barker_rhs = 36.0 * 3600.0 / barker_scale
204     parabolic_anomaly = brentq(
205         lambda anomaly: anomaly + anomaly**3 / 3.0 - barker_rhs,
206         0.0,
207         np.cbrt(3.0 * barker_rhs) + 1.0,
208     )
209     radius_36_hours = radius_perigee * (1.0 + parabolic_anomaly**2)
210     write_output(
211         "07a",
212         [
213             f"time from -90 deg to +90 deg = {time_minus_90_to_plus_90:.6f} s ",
214             f"= {time_minus_90_to_plus_90 / 60.0:.6f} min",
215         ],
216     )
217     write_output(
218         "07b",
219         [
220             f"D(36 h) = {parabolic_anomaly:.9f}",
221             f"radius at 36 h = {radius_36_hours:.6f} km",
222         ],
223     )
224
225
226 def problem_08() -> None:
227     """Curtis 3.16: hyperbolic time of flight and 24-hour radius."""
228     heading("p08")

```

```

229     radius_perigee = 6600.0
230     eccentricity = 2.0 * 1.2**2 - 1.0
231     semimajor_axis_magnitude = radius_perigee / (eccentricity - 1.0)
232     mean_motion = np.sqrt(MU_EARTH / semimajor_axis_magnitude**3)
233     mean_at_90 = hyperbolic_mean_from_true(np.pi / 2.0, eccentricity)
234     coast_time = 2.0 * mean_at_90 / mean_motion
235     mean_anomaly_24_hours = mean_motion * 24.0 * 3600.0
236     anomaly_24_hours = hyperbolic_anomaly(mean_anomaly_24_hours, eccentricity)
237     radius_24_hours = semimajor_axis_magnitude * (
238         eccentricity * np.cosh(anomaly_24_hours) - 1.0
239     )
240     common = [f"e = {eccentricity:.9f}; |a| = {semimajor_axis_magnitude:.6f} km"]
241     write_output(
242         "08a",
243         common
244         + [
245             f"time from -90 deg to +90 deg = {coast_time:.6f} s "
246             f"= {coast_time / 60.0:.6f} min",
247         ],
248     )
249     write_output(
250         "08b",
251         [
252             f"radius at 24 h = {radius_24_hours:.6f} km",
253         ],
254     )
255
256
257 def problem_09() -> None:
258     """Curtis 3.17: propagate from a specified inbound speed to 5 p.m."""
259     heading("p09")
260     radius_perigee = R_EARTH + 200.0
261     eccentricity = 2.0 * 1.1**2 - 1.0
262     semimajor_axis_magnitude = radius_perigee / (eccentricity - 1.0)
263     mean_motion = np.sqrt(MU_EARTH / semimajor_axis_magnitude**3)
264     speed_initial = 8.0
265     radius_initial = 2.0 / (
266         speed_initial**2 / MU_EARTH - 1.0 / semimajor_axis_magnitude
267     )
268     anomaly_magnitude = np.arccosh(
269         (radius_initial / semimajor_axis_magnitude + 1.0) / eccentricity
270     )
271     anomaly_initial = -anomaly_magnitude
272     mean_initial = eccentricity * np.sinh(anomaly_initial) - anomaly_initial
273     mean_final = mean_initial + mean_motion * 7.0 * 3600.0
274     anomaly_final = hyperbolic_anomaly(mean_final, eccentricity)

```

```

275     radius_final = semimajor_axis_magnitude * (
276         eccentricity * np.cosh(anomaly_final) - 1.0
277     )
278     write_output(
279         "09",
280         [
281             f"e = {eccentricity:.9f}; |a| = {semimajor_axis_magnitude:.6f} km",
282             f"radius at 10 a.m. = {radius_initial:.6f} km",
283             f"time from 10 a.m. to perigee = {-mean_initial / mean_motion / 3600.0:.9f} h",
284             f"radius at 5 p.m. = {radius_final:.6f} km",
285             f"altitude at 5 p.m. = {radius_final - R_EARTH:.6f} km",
286         ],
287     )
288
289
290 def problem_10() -> None:
291     """Curtis 3.18: impact test and time from sighting to impact."""
292     heading("p10")
293     radius_initial = R_EARTH + 100000.0
294     speed_initial = 6.0
295     flight_path_angle = np.radians(-80.0)
296     angular_momentum = radius_initial * speed_initial * np.cos(flight_path_angle)
297     specific_energy = speed_initial**2 / 2.0 - MU_EARTH / radius_initial
298     eccentricity = np.sqrt(
299         1.0 + 2.0 * specific_energy * angular_momentum**2 / MU_EARTH**2
300     )
301     semilatus_rectum = angular_momentum**2 / MU_EARTH
302     semimajor_axis_magnitude = MU_EARTH / (2.0 * specific_energy)
303     true_initial_magnitude = np.arccos(
304         (semilatus_rectum / radius_initial - 1.0) / eccentricity
305     )
306     true_initial = -true_initial_magnitude
307     radius_perigee = semilatus_rectum / (1.0 + eccentricity)
308     mean_initial = hyperbolic_mean_from_true(true_initial, eccentricity)
309     mean_motion = np.sqrt(MU_EARTH / semimajor_axis_magnitude**3)
310     if radius_perigee <= R_EARTH:
311         true_event = -np.arccos(
312             (semilatus_rectum / R_EARTH - 1.0) / eccentricity
313         )
314         mean_event = hyperbolic_mean_from_true(true_event, eccentricity)
315         outcome = "impact"
316     else:
317         mean_event = 0.0
318         outcome = "flyby"
319     time_to_event = (mean_event - mean_initial) / mean_motion
320     common = [

```

```
321     f"e = {eccentricity:.9f}; |a| = {semimajor_axis_magnitude:.6f} km",
322     f"perigee radius = {radius_perigee:.6f} km",
323     f"perigee altitude = {radius_perigee - R_EARTH:.6f} km",
324 ]
325 write_output("10a", common + [f"trajectory outcome = {outcome}"])
326 write_output(
327     "10b",
328     [
329         f"time to closest approach = {time_to_event:.6f} s "
330         f"= {time_to_event / 3600.0:.9f} h",
331     ],
332 )
333
334
335 def main() -> None:
336     problem_02()
337     problem_03()
338     problem_04()
339     problem_05()
340     problem_06()
341     problem_07()
342     problem_08()
343     problem_09()
344     problem_10()
345
346
347 if __name__ == "__main__":
348     main()
```